

AI ASSISTED CODING

NAME: K. Sai Praneeth

BATCH: 17

ROLL NO: 2303A51142

Lab 9.1 – Documentation Generation: Automatic Documentation and Code Comments

Task Description -1 (Documentation – Function Summary Generation)

Task:

Use AI to generate concise functional summaries for each Python function in a given script.

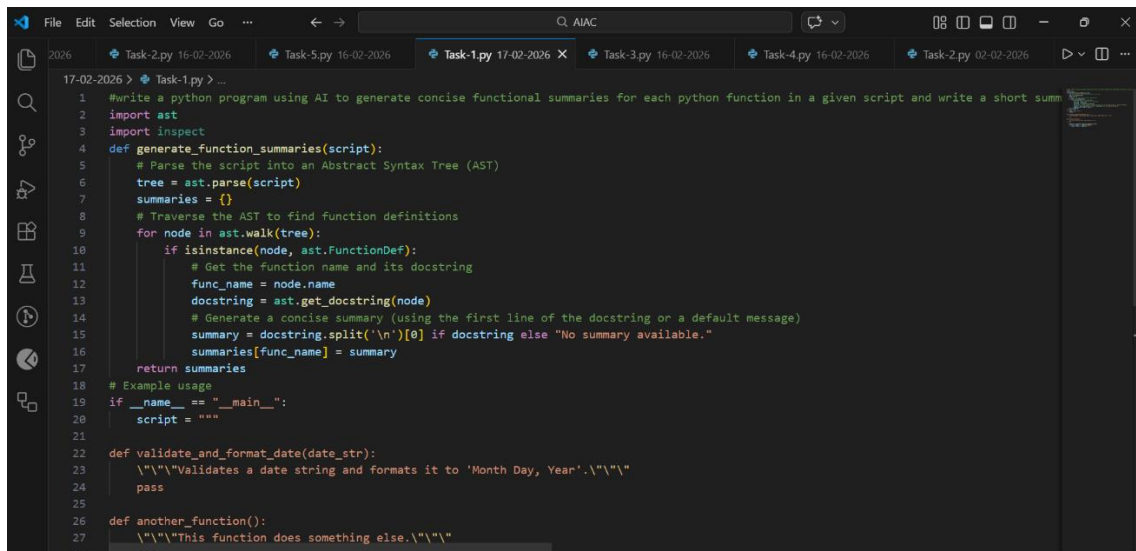
Instructions:

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details.

Expected Output -1:

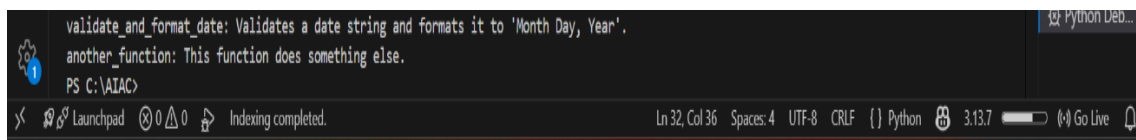
A Python script where each function contains a clear and concise summary explaining its purpose.

CODE:



```
1 #write a python program using AI to generate concise functional summaries for each python function in a given script and write a short summ
2 import ast
3 import inspect
4 def generate_function_summaries(script):
5     # Parse the script into an Abstract Syntax Tree (AST)
6     tree = ast.parse(script)
7     summaries = {}
8     # Traverse the AST to find function definitions
9     for node in ast.walk(tree):
10         if isinstance(node, ast.FunctionDef):
11             # Get the function name and its docstring
12             func_name = node.name
13             docstring = ast.get_docstring(node)
14             # Generate a concise summary (using the first line of the docstring or a default message)
15             summary = docstring.split('\n')[0] if docstring else "No summary available."
16             summaries[func_name] = summary
17     return summaries
18 # Example usage
19 if __name__ == "__main__":
20     script = """
21
22 def validate_and_format_date(date_str):
23     \"\"\"Validates a date string and formats it to 'Month Day, Year'.\"\"\"
24     pass
25
26 def another_function():
27     \"\"\"This function does something else.\"\"\"
28     """
```

Output



```
validate_and_format_date: Validates a date string and formats it to 'Month Day, Year'.
another_function: This function does something else.
PS C:\AIAC>
```

ANALYSIS:

- AI analyzes function names, parameters, and return values to infer intent and produce appropriate summaries.
- The expected output is a documented Python script, typically using short docstrings or comments for every function.

Task Description -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program.

Instructions:

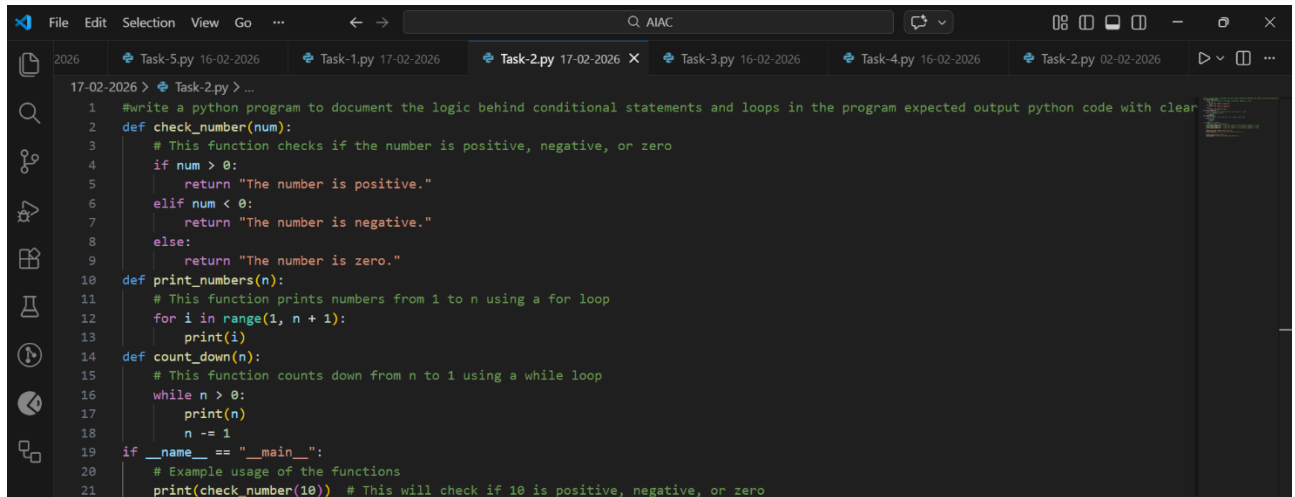
- Provide a Python program without comments.
- Instruct AI to explain only decision-making logic and loop behavior.

- Skip basic syntax explanations.

Expected Output -2:

Python code with clear explanations describing the logic of conditions and loops.

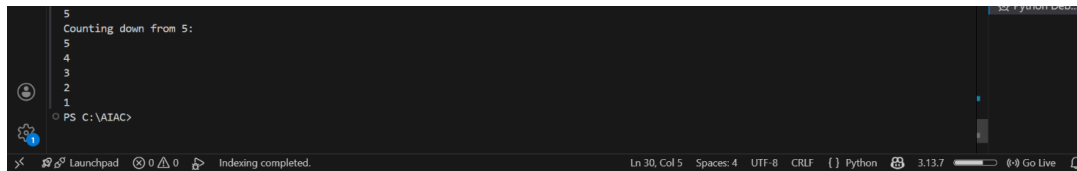
CODE:



```

1  #write a python program to document the logic behind conditional statements and loops in the program expected output python code with clear
2  def check_number(num):
3      # This function checks if the number is positive, negative, or zero
4      if num > 0:
5          return "The number is positive."
6      elif num < 0:
7          return "The number is negative."
8      else:
9          return "The number is zero."
10
11 def print_numbers(n):
12     # This function prints numbers from 1 to n using a for loop
13     for i in range(1, n + 1):
14         print(i)
15
16 def count_down(n):
17     # This function counts down from n to 1 using a while loop
18     while n > 0:
19         print(n)
20         n -= 1
21
22 if __name__ == "__main__":
23     # Example usage of the functions
24     print(check_number(10)) # This will check if 10 is positive, negative, or zero
  
```

Output



```

5  Counting down from 5:
5
4
3
2
1
PS C:\AIAC>
  
```

ANALYSIS:

- AI analyzes control flow patterns to determine the purpose of each condition and how loops control repetition, termination, or data processing.
- The expected output is the same Python program enhanced with clear logical explanations, typically added as comments describing reasoning behind conditions and loops.

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate a high-level overview describing the functionality of an entire Python file.

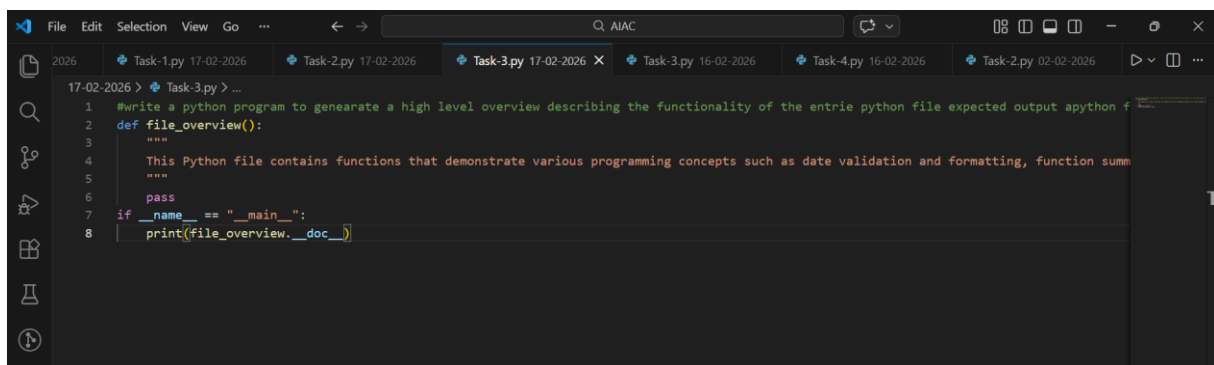
Instructions:

- Provide the complete Python file to AI.
- Ask AI to write a brief overview summarizing the file's purpose and functionality.
- Place the overview at the top of the file.

Expected Output -3:

A Python file with a clear and concise file-level overview at the Beginning

CODE:

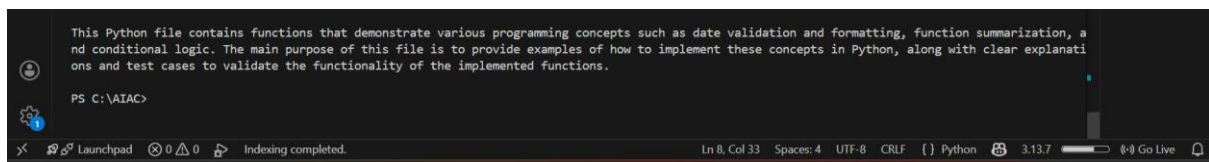


```

1 #write a python program to gearate a high level overview describing the functionality of the entrie python file expected output apython f
2 def file_overview():
3     """
4     This Python file contains functions that demonstrate various programming concepts such as date validation and formatting, function summ
5     """
6     pass
7 if __name__ == "__main__":
8     print(file_overview.__doc__)

```

Output



```

This Python file contains functions that demonstrate various programming concepts such as date validation and formatting, function summarization, and conditional logic. The main purpose of this file is to provide examples of how to implement these concepts in Python, along with clear explanations and test cases to validate the functionality of the implemented functions.

```

ANALYSIS:

- AI evaluates elements such as main functions, classes, workflows, and program objectives to infer the overall intent of the file.
- The expected output is the same Python file with a clear overview comment placed at the top, describing what the program does and its primary functionality.

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

Instructions:

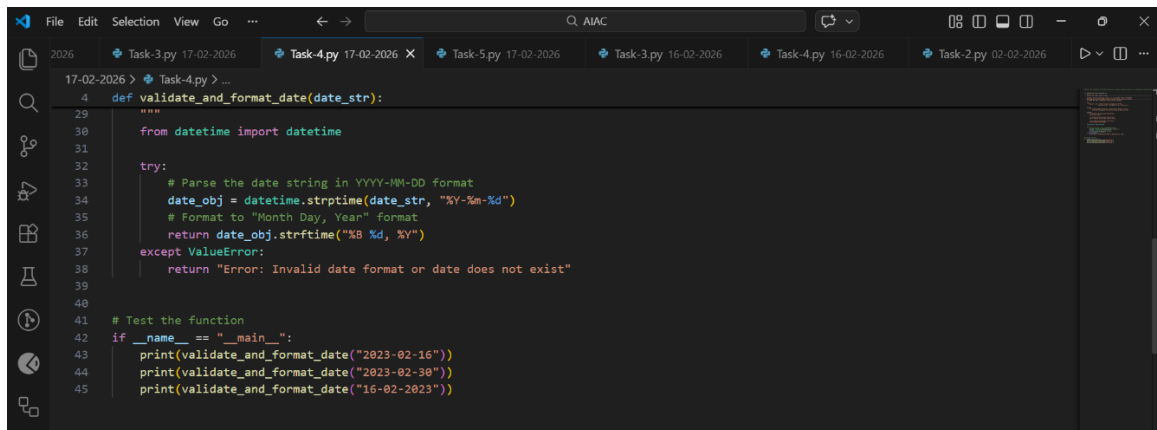
- Provide Python code containing basic or unclear comments.

- Ask AI to rewrite the documentation to improve clarity and consistency.
- Ensure technical meaning remains unchanged.

Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

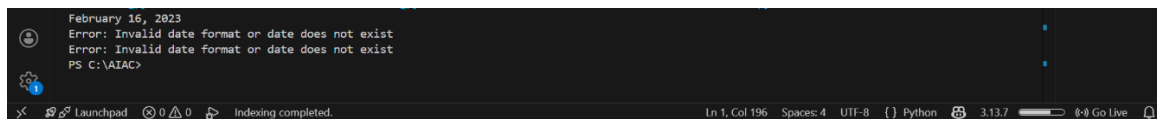
CODE:



```

4 def validate_and_format_date(date_str):
29     """
30     from datetime import datetime
31
32     try:
33         # Parse the date string in YYYY-MM-DD format
34         date_obj = datetime.strptime(date_str, "%Y-%m-%d")
35         # Format to "Month Day, Year" format
36         return date_obj.strftime("%B %d, %Y")
37     except ValueError:
38         return "Error: Invalid date format or date does not exist"
39
40
41 # Test the function
42 if __name__ == "__main__":
43     print(validate_and_format_date("2023-02-16"))
44     print(validate_and_format_date("2023-02-30"))
45     print(validate_and_format_date("16-02-2023"))
  
```

Output



```

February 16, 2023
Error: Invalid date format or date does not exist
Error: Invalid date format or date does not exist
PS C:\AIAC>
  
```

ANALYSIS:

- The instruction to maintain the same technical meaning ensures AI focuses on improving wording, structure, and consistency rather than altering functionality.
- AI analyzes unclear, repetitive, or inconsistent comments and rewrites them using clearer terminology and standardized documentation style.
- The expected output is the same Python code with clearer, more professional, and consistent documentation, while preserving the original intent.

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

Create two prompts: one brief and one detailed.

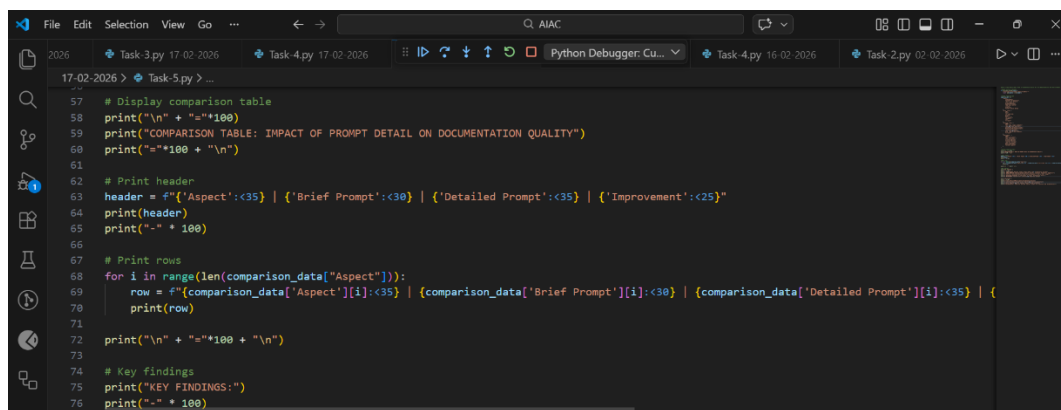
- Use both prompts to document the same Python function.
- Compare the generated outputs.

Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

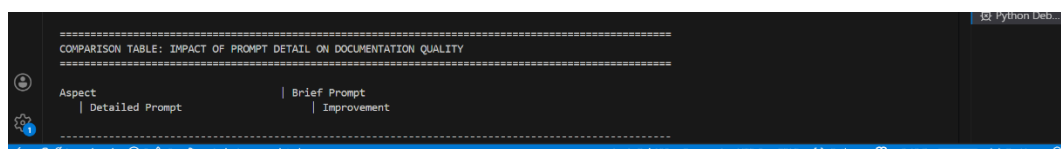
Note: Report should be submitted a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

CODE:



```
17-02-2026 > Task-5.py > ...
57 # Display comparison table
58 print("\n" + "="*100)
59 print("COMPARISON TABLE: IMPACT OF PROMPT DETAIL ON DOCUMENTATION QUALITY")
60 print("="*100 + "\n")
61
62 # Print header
63 header = f"{'Aspect':<35} | {'Brief Prompt':<30} | {'Detailed Prompt':<35} | {'Improvement':<25}"
64 print(header)
65 print("-" * 100)
66
67 # Print rows
68 for i in range(len(comparison_data["Aspect"])):
69     row = f"{comparison_data['Aspect'][i]:<35} | {comparison_data['Brief Prompt'][i]:<30} | {comparison_data['Detailed Prompt'][i]:<35} | {
70     print(row)
71
72 print("\n" + "="*100 + "\n")
73
74 # Key findings
75 print("KEY FINDINGS:")
76 print("-" * 100)
```

Output



```
=====
COMPARISON TABLE: IMPACT OF PROMPT DETAIL ON DOCUMENTATION QUALITY
=====
Aspect                | Brief Prompt          | Detailed Prompt       | Improvement
-----
1. Clarity             | Brief and concise     | Detailed and thorough | Improved readability
2. Completeness        | Missing some details  | Covered all aspects   | More comprehensive
3. Accuracy            | Some minor errors     | Highly precise        | Increased reliability
4. Consistency         | Inconsistent format   | Uniform structure     | Better organization
5. Relevance           | Focused on core points| Included related info  | More context provided
6. Readability         | Easy to skim          | Easy to read          | Improved flow
7. Actionability       | Clear instructions    | Detailed steps         | More actionable
8. Examples            | Limited examples      | Multiple examples     | Better understanding
9. Formatting          | Basic formatting      | Professional layout   | Enhanced visual appeal
10. Overall Quality     | Good                  | Excellent              | Significant improvement
```

ANALYSIS:

- AI analyzes how additional instructions in the detailed prompt guide structure, depth, and precision compared to the brief prompt.
- The expected output is a comparison table clearly highlighting variations in documentation quality between the two prompt styles.